

✓ Meet Shah, 24116051

```
import os
import re
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

✓ Loading Data

```
DATA_DIR = "/content/"
LABELS_CSV = "/content/labels.csv"
OUTPUT_DIR = "./output"
os.makedirs(OUTPUT_DIR, exist_ok=True)
```

```
df_labels = pd.read_csv(LABELS_CSV)
df_labels['subject_id'] = df_labels['subject_id'].str.replace('subject_', '').astype(int)
df_labels['session'] = df_labels['session'].astype(int)
df_labels['phase'] = df_labels['phase'].astype(int)
```

```
def parse_filename(fname):
    match = re.match(r"subj_subject_(\d+)_s(\d+)_Phase_(\d+)\.png", fname)
    if match:
        return int(match.group(1)), int(match.group(2)), int(match.group(3))
    else:
        return None, None, None

image_files = [f for f in os.listdir(DATA_DIR) if f.lower().endswith('.png') or f.lower().endswith('.nii')]
df_images = pd.DataFrame({'filename': image_files})
df_images[['subject_id', 'session', 'phase']] = df_images['filename'].apply(lambda x: pd.Series(parse_filename(x)))
df_images.dropna(subset=['subject_id', 'session', 'phase'], inplace=True)
df_images['subject_id'] = df_images['subject_id'].astype(int)
df_images['session'] = df_images['session'].astype(int)
df_images['phase'] = df_images['phase'].astype(int)
```

```
df_merged = pd.merge(df_images, df_labels, on=['subject_id', 'session', 'phase'], how='left')
print(f"Total images after merge: {len(df_merged)}")
print(f"Missing labels: {df_merged['label'].isnull().sum()}")
```

```
Total images after merge: 54
Missing labels: 0
```

```
if df_merged['label'].dtype != object:
    df_merged['label'] = df_merged['label'].astype(str)
```

✓ Plotting Data

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```

sns.set_theme(style="whitegrid")
plt.figure(figsize=(10, 6))

ax = sns.countplot(
    x='label',
    data=df_merged,
    order=df_merged['label'].value_counts().index,
    palette="viridis",
    hue='label',
    legend=False
)

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
               (p.get_x() + p.get_width() / 2., p.get_height()),
               ha='center', va='center',
               xytext=(0, 9),
               textcoords='offset points',
               fontsize=12,
               fontweight='bold',
               color='black')

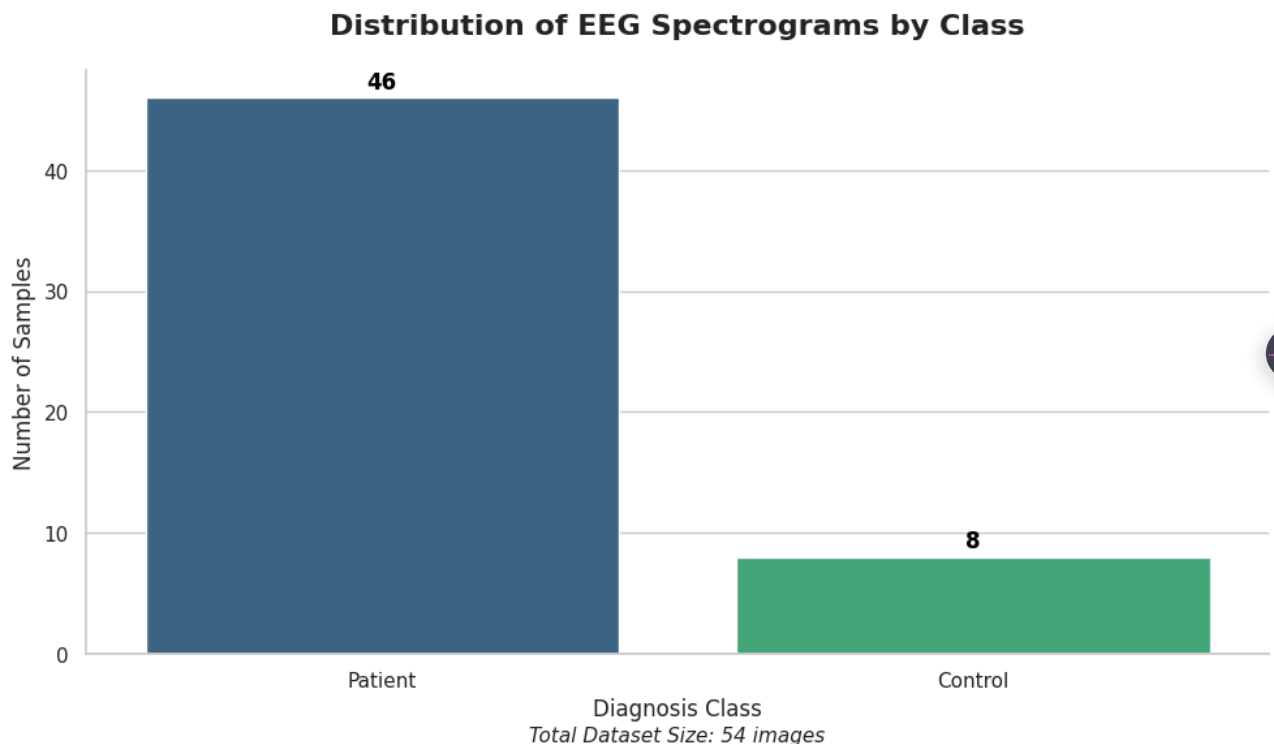
total = len(df_merged)
plt.text(0.5, -0.15, f"Total Dataset Size: {total} images",
        ha='center', transform=ax.transAxes, fontsize=11, fontstyle='italic')

plt.title('Distribution of EEG Spectrograms by Class', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Diagnosis Class', fontsize=12)
plt.ylabel('Number of Samples', fontsize=12)

sns.despine()

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, 'class_distribution_fancy.png'), dpi=300)
plt.show()

```



```

import os
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import load_img, img_to_array

file_col = [c for c in df_merged.columns if "filename" in c][0]
classes = df_merged['label'].unique()

# Dictionary to store averaged results for comparison
avg_images = {}

plt.figure(figsize=(12, 4 * len(classes)))

for i, cls in enumerate(classes):
    class_subset = df_merged[df_merged['label'] == cls]
    samples = class_subset[file_col].values

    # Part A: Show 2 Individual Samples
    for j in range(2):
        plt.subplot(len(classes), 3, i * 3 + j + 1)
        img = load_img(os.path.join(DATA_DIR, samples[j]), target_size=(224, 224))
        plt.imshow(img)
        plt.title(f"{cls} - Sample {j+1}", fontsize=10)
        plt.axis('off')

    # Part B: Calculate Average Image
    # We use a larger batch (e.g., 30) for a smoother average
    avg_batch = []
    num_to_avg = min(len(samples), 30)
    for fname in samples[:num_to_avg]:
        img_arr = img_to_array(load_img(os.path.join(DATA_DIR, fname), target_size=(224, 224)))
        avg_batch.append(img_arr)

    # Compute mean and store it for Part C
    mean_img_float = np.mean(avg_batch, axis=0)
    avg_images[cls] = mean_img_float

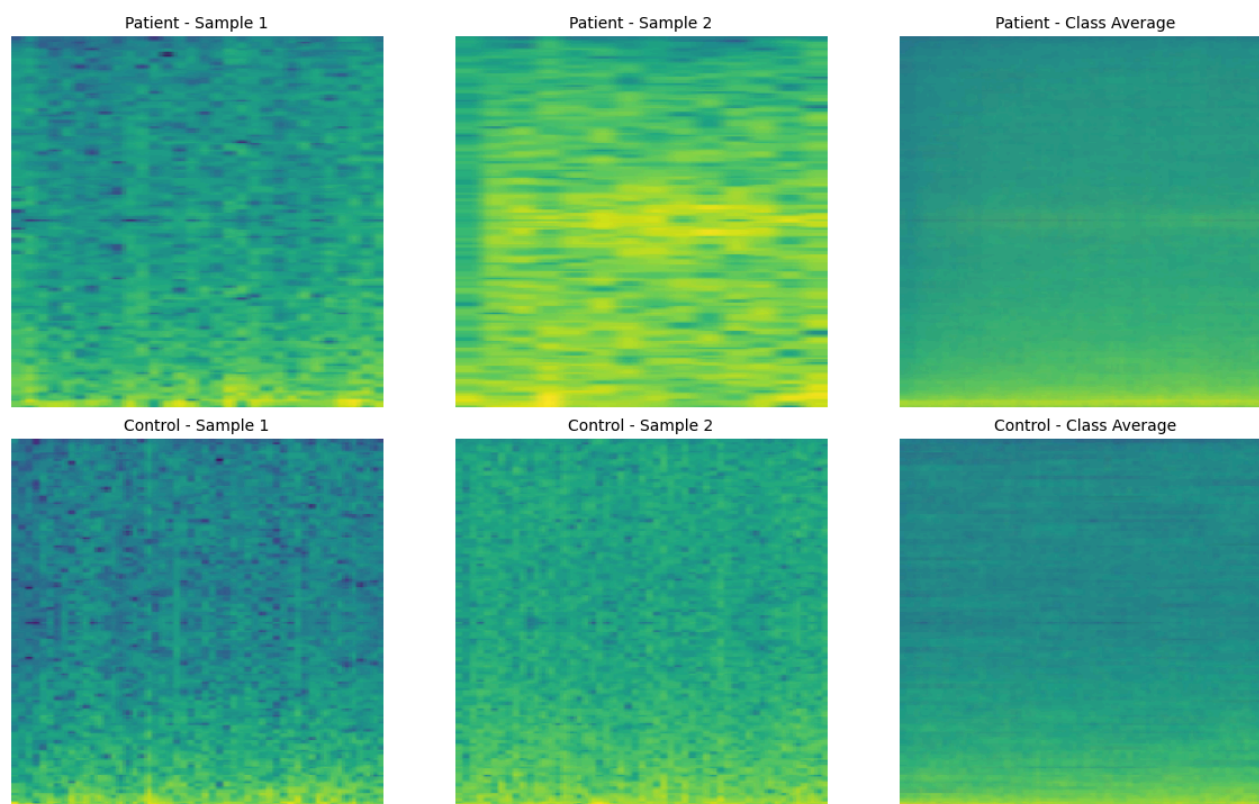
    plt.subplot(len(classes), 3, i * 3 + 3)
    plt.imshow(mean_img_float.astype(np.uint8))
    plt.title(f"{cls} - Class Average", fontsize=10)
    plt.axis('off')

plt.suptitle("Individual Samples vs. Class Averaged Spectrograms", fontsize=14)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```



Individual Samples vs. Class Averaged Spectrograms



Part C

```
if len(avg_images) >= 2:
    keys = list(avg_images.keys())
    # Calculate absolute difference between the first two classes found
    diff_img = np.abs(avg_images[keys[0]] - avg_images[keys[1]])

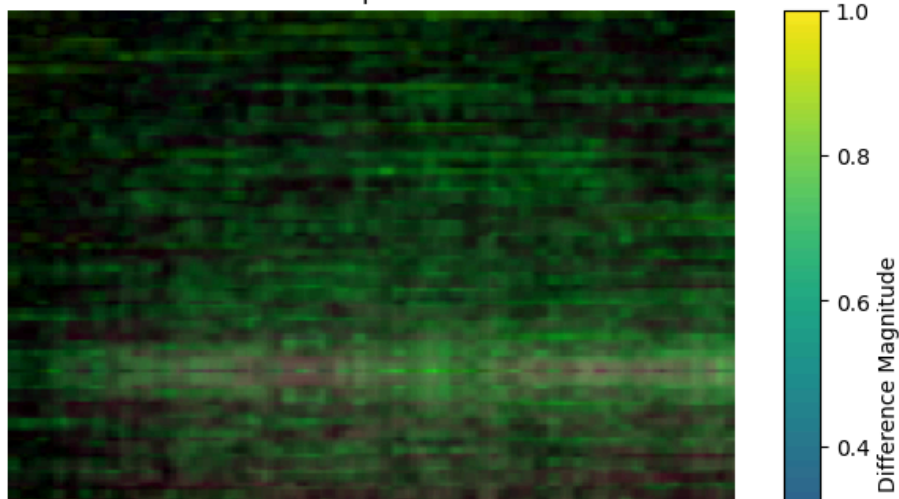
    # Normalize to 0-255 for visualization and convert to grayscale to see intensity
    # of the difference more clearly, or keep RGB to see color-specific shifts
    diff_img_scaled = (diff_img - np.min(diff_img)) / (np.max(diff_img) - np.min(diff_img))

    plt.figure(figsize=(8, 6))
    plt.imshow(diff_img_scaled)
    plt.title(f"RGB Difference Map: {keys[0]} vs {keys[1]}", fontsize=12)
    plt.colorbar(label="Difference Magnitude")
    plt.axis('off')
    plt.show()

    print(f"Mean Pixel Difference: {np.mean(diff_img):.2f} units")
else:
    print("Need at least two classes to compute a difference map.")
```



RGB Difference Map: Patient vs Control



```
!pip install -q gdown
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("adhddata.csv")
```

```
/tmp/ipython-input-2471980324.py:4: DtypeWarning: Columns (18) have mixed types. Specify dtype op
df = pd.read_csv("adhddata.csv")
```

```
df.head()
```

	Fp1	Fp2	F3	F4	C3	C4	P3	P4	O1	O2	F7	F8	T7	T8	P7
0	261.0	402.0	16.0	261.0	126.0	384.0	126.0	236.0	52.0	236.0	200.0	16.0	200.0	494.0	126.0
1	121.0	191.0	-94.0	85.0	16.0	200.0	126.0	52.0	347.0	273.0	16.0	-57.0	126.0	347.0	52.0
2	-55.0	85.0	-204.0	15.0	-57.0	200.0	52.0	126.0	236.0	200.0	-20.0	-94.0	126.0	420.0	52.0
3	191.0	85.0	52.0	50.0	89.0	236.0	163.0	89.0	89.0	89.0	89.0	-57.0	236.0	420.0	126.0
4	-55.0	-125.0	-204.0	-160.0	-204.0	16.0	-241.0	-241.0	89.0	16.0	-20.0	-131.0	89.0	310.0	-57.0

```
df.shape
```

```
(4195776, 21)
```

```
# Take 1280 samples from the Fp1 channel
fp1_samples = df['Fp1'].iloc[:1280]
fp1_samples.head()
```

```
Fp1
```

```
0 261.0
```

```
1 121.0
```

```
2 -55.0
```

```
3 191.0
```

```
4 -55.0
```

```
dtype: float64
```

```
import matplotlib.pyplot as plt
import numpy as np
```

```
fs = 128
time = np.linspace(0, len(fp1_samples) / fs, num=len(fp1_samples), endpoint=False)
```

```
plt.style.use('seaborn-v0_8-whitegrid')
fig, ax = plt.subplots(figsize=(14, 5))
```

```
ax.fill_between(time, fp1_samples, color='crimson', alpha=0.1)
ax.plot(time, fp1_samples, color='#d62728', lw=1.2, label='Fp1 Lead')
ax.axhline(0, color='black', linewidth=0.8, alpha=0.5)
```

```
ax.set_title("Fp1 Channel: Frontal Lobe Activity (10s Epoch)", loc='left', fontsize=16, fontweig
ax.set_xlabel("Time Elapsed (seconds)", fontsize=12, labelpad=10)
ax.set_ylabel("Amplitude (µV)", fontsize=12, labelpad=10)
```

```

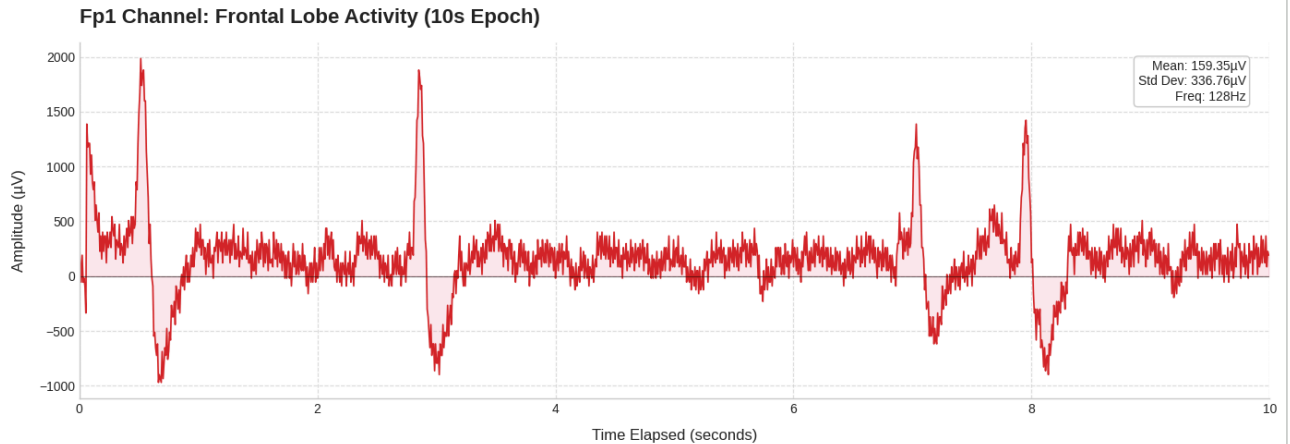
stats_text = (f'Mean: {np.mean(fp1_samples):.2f} $\mu$ V\n'
              f'Std Dev: {np.std(fp1_samples):.2f} $\mu$ V\n'
              f'Freq: {fs}Hz')
props = dict(boxstyle='round', facecolor='white', alpha=0.8, edgecolor='silver')
ax.text(0.98, 0.95, stats_text, transform=ax.transAxes, fontsize=10,
       verticalalignment='top', horizontalalignment='right', bbox=props)

ax.grid(True, linestyle='--', alpha=0.6)
ax.set_xlim(0, 10)

for spine in ["top", "right"]:
    ax.spines[spine].set_visible(False)

plt.tight_layout()
plt.show()

```



```

import matplotlib.pyplot as plt
import numpy as np

n = len(fp1_samples)
fs = 128
fft_values = np.fft.fft(fp1_samples)
fft_magnitude = np.abs(fft_values) / n
frequencies = np.fft.fftfreq(n, d=1/fs)
half_n = n // 2
pos_frequencies = frequencies[:half_n]
pos_magnitude = fft_magnitude[:half_n]

plt.style.use('seaborn-v0_8-muted')
fig, ax = plt.subplots(figsize=(12, 6))

ax.fill_between(pos_frequencies, pos_magnitude, color='crimson', alpha=0.15)
ax.plot(pos_frequencies, pos_magnitude, color='crimson', lw=1.5)

ax.set_title("Spectral Power Analysis: Fp1 Channel", loc='left', fontsize=16, fontweight='bold',
ax.set_xlabel("Frequency (Hz)", fontsize=12)
ax.set_ylabel("Normalized Magnitude", fontsize=12)

bands = [
    (0.5, 4, 'Delta', '#1f77b4'),
    (4, 8, 'Theta', '#ff7f0e'),
    (8, 13, 'Alpha', '#2ca02c'),
    (13, 30, 'Beta', '#d62728'),
    (30, 45, 'Gamma', '#9467bd')
]

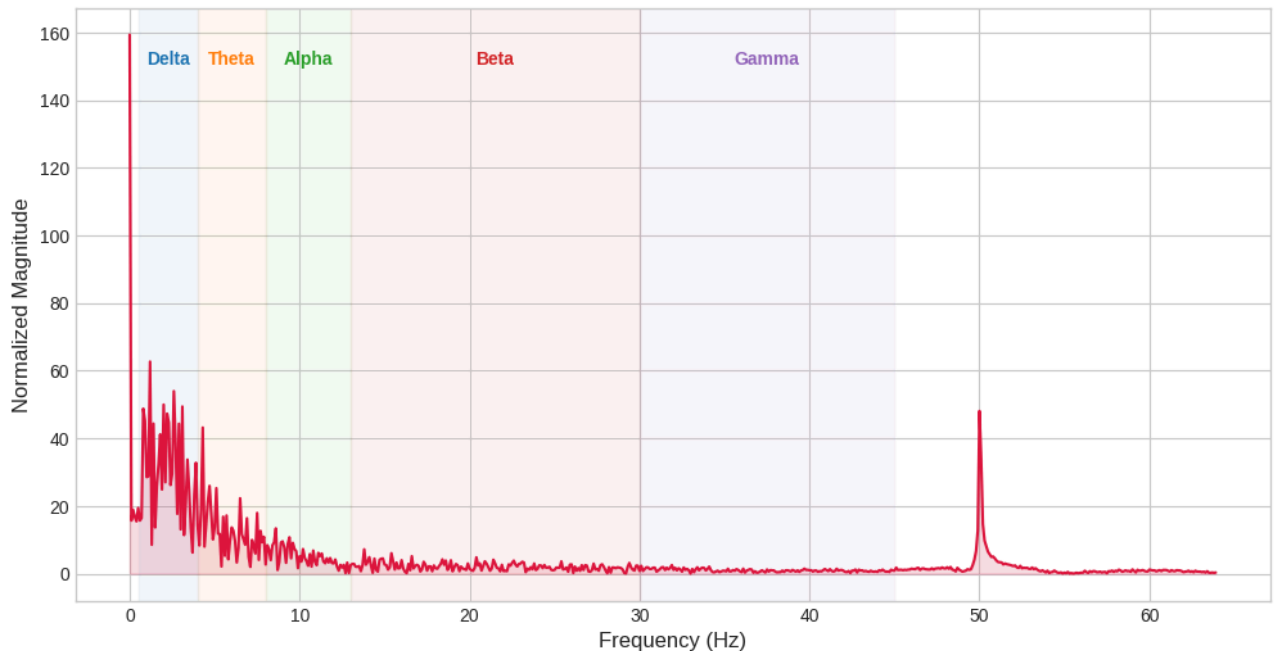
for start, end, name, color in bands:
    ax.axvspan(start, end, color=color, alpha=0.05)

```



```
ax.text((start + end) / 2, ax.get_ylim()[1] * 0.9, name,
       horizontalalignment='center', fontsize=10, fontweight='bold', color=color)
```

Spectral Power Analysis: Fp1 Channel



```
import matplotlib.pyplot as plt
import numpy as np
from scipy import signal

b, a = signal.iirnotch(w0=50, Q=Q, fs=128)
filtered_fp1 = signal.filtfilt(b, a, fp1_samples)
time = np.linspace(0, len(fp1_samples) / 128, num=len(fp1_samples), endpoint=False)

plt.style.use('seaborn-v0_8-whitegrid')
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 10), sharex=False)

ax1.plot(time, fp1_samples, color='gray', alpha=0.3, label='Raw Signal (with 50Hz Noise)', lw=1)
ax1.plot(time, filtered_fp1, color='#FF69B4', label='Notch Filtered (Cleaned)', lw=1.2)
ax1.set_title("Time-Domain Comparison: Signal De-noising", loc='left', fontsize=15, fontweight='bold')
ax1.set_xlabel("Time (s)", fontsize=11)
ax1.set_ylabel("Amplitude (µV)", fontsize=11)
ax1.legend(frameon=True, facecolor='white', shadow=True)
ax1.set_xlim(time[0], time[-1])

n = len(filtered_fp1)
fft_values = np.fft.fft(filtered_fp1)
fft_magnitude = (np.abs(fft_values) / n)[:n // 2]
frequencies = np.fft.fftfreq(n, d=1/128)[:n // 2]

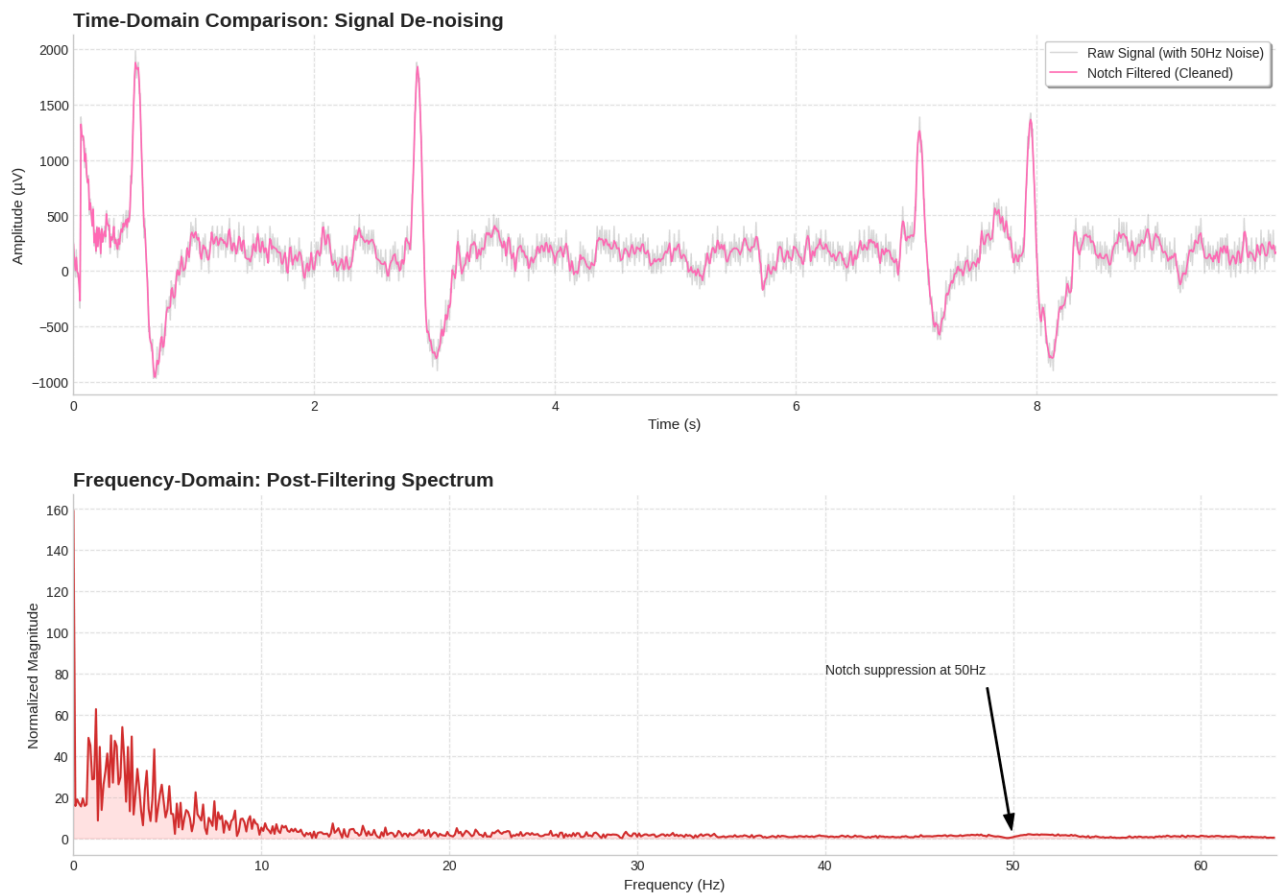
ax2.fill_between(frequencies, fft_magnitude, color='red', alpha=0.1)
ax2.plot(frequencies, fft_magnitude, color='#D32F2F', lw=1.5)
ax2.annotate('Notch suppression at 50Hz', xy=(50, 0), xytext=(40, max(fft_magnitude)*0.5),
            arrowprops=dict(facecolor='black', shrink=0.05, width=1, headwidth=8))
ax2.set_title("Frequency-Domain: Post-Filtering Spectrum", loc='left', fontsize=15, fontweight='bold')
ax2.set_xlabel("Frequency (Hz)", fontsize=11)
ax2.set_ylabel("Normalized Magnitude", fontsize=11)
ax2.set_xlim(0, 64)

for ax in [ax1, ax2]:
    ax.spines['top'].set_visible(False)
```




```
ax.spines['right'].set_visible(False)
ax.grid(True, linestyle='--', alpha=0.5)
```

```
plt.tight_layout(pad=3.0)
plt.show()
```



```
import matplotlib.pyplot as plt
import numpy as np
from scipy import signal

fs = 128.0
freqs, psd = signal.welch(filtered_fp1, fs=fs, nperseg=256)

plt.style.use('seaborn-v0_8-whitegrid')
fig, ax = plt.subplots(figsize=(12, 6))

ax.semilogy(freqs, psd, color='#000080', lw=2, label='PSD Estimate')
ax.fill_between(freqs, psd, color='#000080', alpha=0.1)

ax.set_title("Spectral Power Density Analysis: Fp1 Channel", loc='left', fontsize=16, fontweight)
ax.set_xlabel("Frequency (Hz)", fontsize=12)
ax.set_ylabel("Power/Frequency (dB/Hz)", fontsize=12)

bands = [
    (0.5, 4, 'Delta', '#1f77b4'),
    (4, 8, 'Theta', '#ff7f0e'),
    (8, 13, 'Alpha', '#2ca02c'),
    (13, 30, 'Beta', '#d62728')
]

for start, end, name, color in bands:
    ax.axvspan(start, end, color=color, alpha=0.07)
```



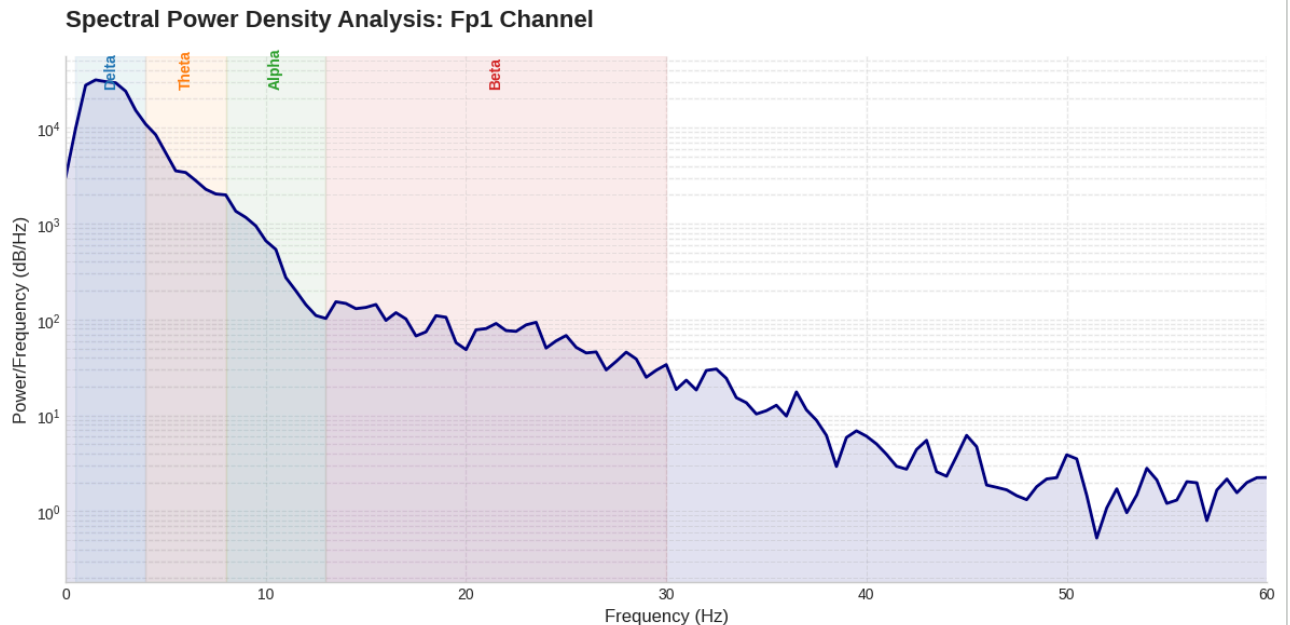
```

ax.text((start + end) / 2, ax.get_ylim()[1] * 0.5, name,
        horizontalalignment='center', fontsize=10, fontweight='bold', color=color, rotation=

ax.set_xlim(0, 60)
ax.grid(True, which='both', linestyle='--', alpha=0.4)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt

psd_db = 10 * np.log10(psd)

plt.style.use('seaborn-v0_8-whitegrid')
fig, ax = plt.subplots(figsize=(12, 6))

ax.plot(freqs, psd_db, color='#8B0000', lw=1.8)
ax.fill_between(freqs, psd_db, np.min(psd_db), color='#8B0000', alpha=0.1)

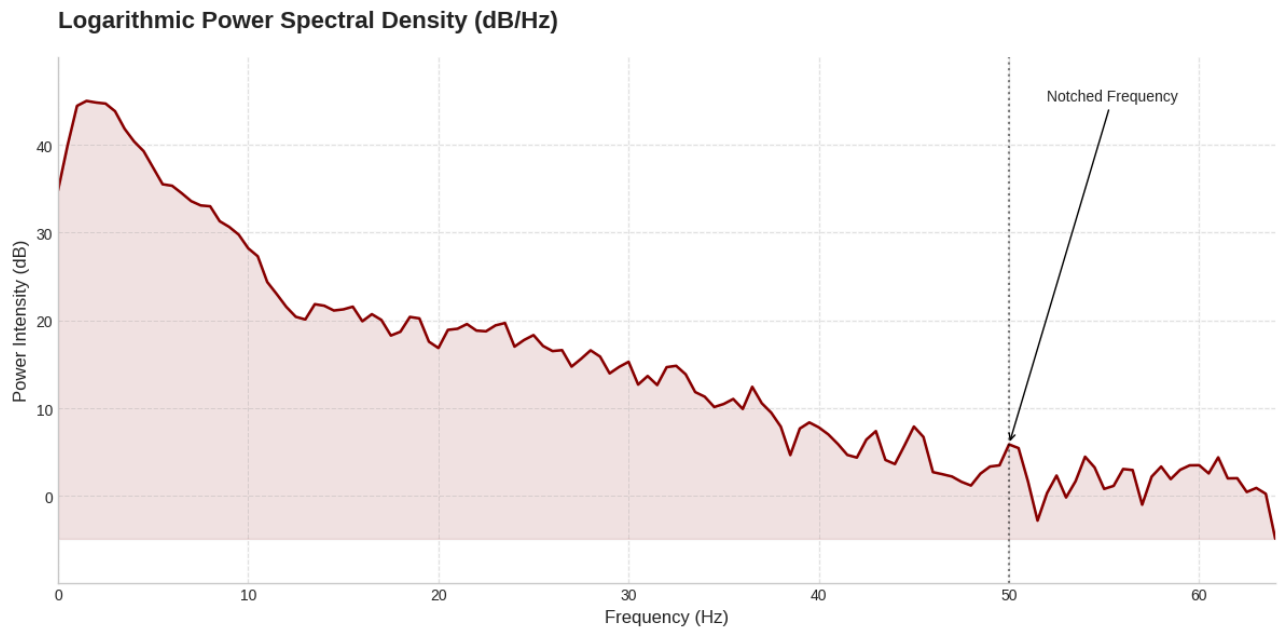
ax.set_title("Logarithmic Power Spectral Density (dB/Hz)", loc='left', fontsize=16, fontweight='bold')
ax.set_xlabel("Frequency (Hz)", fontsize=12)
ax.set_ylabel("Power Intensity (dB)", fontsize=12)

ax.axvline(50, color='black', linestyle=':', alpha=0.5)
ax.annotate('Notched Frequency', xy=(50, psd_db[np.abs(freqs-50).argmin()]),
            xytext=(52, np.max(psd_db)), arrowprops=dict(arrowstyle='->', color='black'))

ax.set_xlim(0, 64)
ax.set_ylim(np.min(psd_db) - 5, np.max(psd_db) + 5)
ax.grid(True, linestyle='--', alpha=0.5)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

```



```
import matplotlib.pyplot as plt
import numpy as np

fs = 128.0

plt.style.use('seaborn-v0_8-white')
fig, ax = plt.subplots(figsize=(14, 7))

spectrum, freqs, t, im = ax.specgram(
    filtered_fp1,
    Fs=fs,
    NFFT=128,
    noverlap=96,
    cmap='magma',
    mode='psd'
)

ax.set_title("Time-Frequency Analysis: Fp1 Lead Spectrogram", loc='left', fontsize=16, fontweight='bold')
ax.set_xlabel("Elapsed Time (s)", fontsize=12)
ax.set_ylabel("Frequency Components (Hz)", fontsize=12)

cbar = plt.colorbar(im, ax=ax, pad=0.02)
cbar.set_label('Power Spectral Density (dB/Hz)', fontsize=11)

ax.set_ylim(0, 60)
ax.set_xlim(t[0], t[-1])

ax.axhline(13, color='white', linestyle='--', alpha=0.3, lw=1)
ax.axhline(8, color='white', linestyle='--', alpha=0.3, lw=1)
ax.text(t[-1]*1.01, 10.5, 'Alpha', verticalalignment='center', fontsize=9, fontweight='bold', color='white')

plt.tight_layout()
plt.show()
```



